



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 08

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 11 de abril de 2026

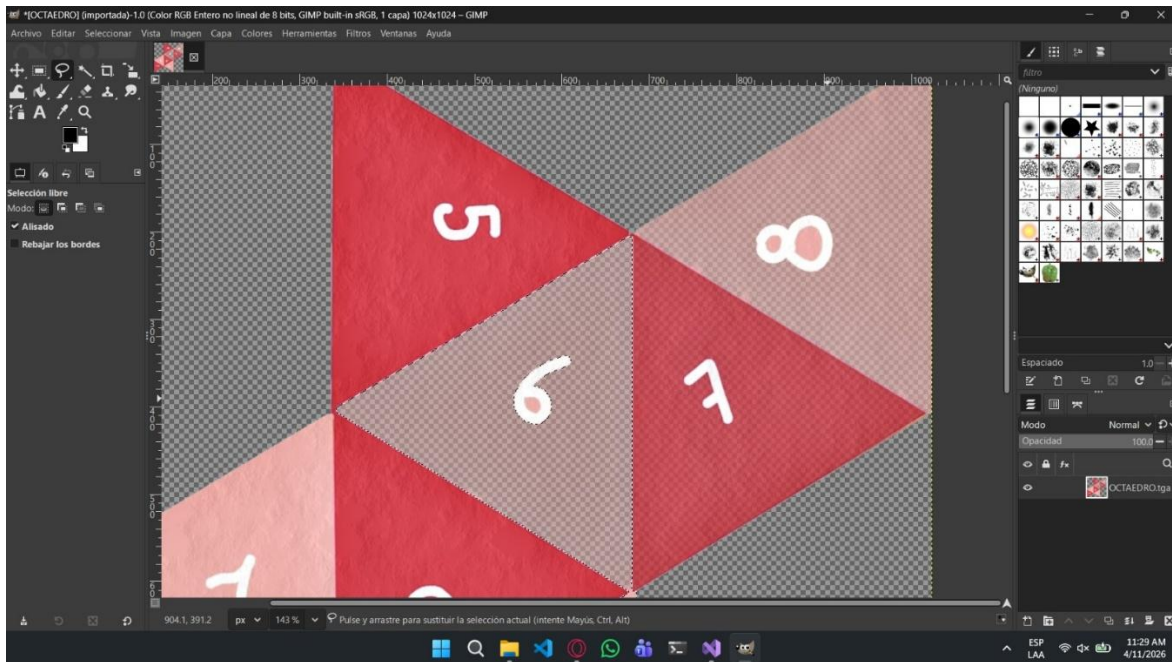
CALIFICACIÓN: _____

EJERCICIOS REALIZADOS

Agregar octaedro y normales

Como primer ejercicio se pidió agregar a nuestro código el octaedro que habíamos realizado en la práctica 6, editando sus normales para que las caras del dado fueran iluminadas correctamente.

Primeramente, se hizo un cambio extra a la textura del dado, ya que se quiso notar la transparencia del mismo se modifico la imagen OCTAEDRO.TGA en GIMP, para colocar la cara del numero 8 con cierta opacidad y la cara numero 6 con aun más opacidad.



Ahora pasaremos a los cambios de nuestro código.

Main

Primeramente, creamos la textura que modificamos anteriormente.

```
Texture OctaedroTexture;
```

Partimos de entender que una normal para nuestro dado es un vector que apunta de manera perpendicular hacia afuera de las caras del octaedro, ya que su función es decir donde puede rebotar la iluminación que colocamos en nuestros escenarios. Entonces, a partir de la interacción con la GeminiIA* (Más abajo en la sección de problemas presentados se adjuntan capturas de la interacción con la misma), modificamos los signos y valores de las normales a $0.577f$ para cada cara. Además como extra mencionado en clase, se puso la cara 8 y 6 hasta el final de la función para asegurar que las otras caras no “desaparecieran”

por el fase culling, sin embargo, esto solo funciona para la cara 8 ya que es la última en “crearse” para la cara 6 seguimos teniendo el detalle que desaparece la cara 8.

```
// =====
void CrearOctaedro()
{
    unsigned int octa_indices[] = {
        // --- MITAD SUPERIOR ---
        0, 1, 2,    // Cara 1: Superior Frontal-Derecha
        3, 4, 5,    // Cara 2: Superior Trasera-Derecha
        6, 7, 8,    // Cara 3: Superior Trasera-Izquierda
        9, 10, 11,  // Cara 4: Superior Frontal-Izquierda

        // --- MITAD INFERIOR ---
        12, 13, 14, // Cara 5: Inferior Frontal-Derecha
        15, 16, 17, // Cara 6: Inferior Trasera-Derecha
        18, 19, 20, // Cara 7: Inferior Trasera-Izquierda
        21, 22, 23  // Cara 8: Inferior Frontal-Izquierda
    };

    GLfloat octa_vertices[] = {
        // x      y      z      S      T      NX      NY      NZ
        // ===== MITAD SUPERIOR (Triángulos apuntando hacia arriba) =====
        // Cara 1: Superior Frontal-Derecha (+, +, +)
        0.0f, 0.5f, 0.0f, 0.32f, 0.20f, 0.577f, 0.577f, 0.577f,
        0.0f, 0.0f, 0.5f, 0.01f, 0.39f, 0.577f, 0.577f, 0.577f,
        0.5f, 0.0f, 0.0f, 0.01f, 0.01f, 0.577f, 0.577f, 0.577f,

        // Cara 2: Superior Trasera-Derecha (+, +, -)
        0.0f, 0.5f, 0.0f, 0.33f, 0.21f, 0.577f, 0.577f, -0.577f,
        0.5f, 0.0f, 0.0f, 0.33f, 0.59f, 0.577f, 0.577f, -0.577f,
        0.0f, 0.0f, -0.5f, 0.01f, 0.41f, 0.577f, 0.577f, -0.577f,

        // Cara 3: Superior Trasera-Izquierda (-, +, -)
        0.0f, 0.5f, 0.0f, 0.34f, 0.21f, -0.577f, 0.577f, -0.577f,
        0.0f, 0.0f, -0.5f, 0.65f, 0.40f, -0.577f, 0.577f, -0.577f,
        -0.5f, 0.0f, 0.0f, 0.34f, 0.59f, -0.577f, 0.577f, -0.577f,

        // Cara 4: Superior Frontal-Izquierda (-, +, +)
        0.0f, 0.5f, 0.0f, 0.35f, 0.21f, -0.577f, 0.577f, 0.577f,
        -0.5f, 0.0f, 0.0f, 0.67f, 0.01f, -0.577f, 0.577f, 0.577f,
        0.0f, 0.0f, 0.5f, 0.67f, 0.40f, -0.577f, 0.577f, 0.577f,

        // ===== MITAD INFERIOR (Triángulos apuntando hacia abajo) =====

        // Cara 5: Inferior Frontal-Izquierda (-, -, +)
        0.0f, -0.5f, 0.0f, 0.66f, 0.80f, -0.577f, -0.577f, 0.577f,
        0.0f, 0.0f, 0.5f, 0.33f, 0.99f, -0.577f, -0.577f, 0.577f,
        -0.5f, 0.0f, 0.0f, 0.33f, 0.61f, -0.577f, -0.577f, 0.577f,

        // Cara 7: Inferior Trasera-Derecha (+, -, -)
        0.0f, -0.5f, 0.0f, 0.67f, 0.79f, 0.577f, -0.577f, -0.577f,
        0.0f, 0.0f, -0.5f, 0.67f, 0.41f, 0.577f, -0.577f, -0.577f,
        0.5f, 0.0f, 0.0f, 0.98f, 0.60f, 0.577f, -0.577f, -0.577f,

        // Cara 6: Inferior Frontal-Derecha (+, -, +)
        0.0f, -0.5f, 0.0f, 0.66f, 0.79f, 0.577f, -0.577f, 0.577f,
        0.5f, 0.0f, 0.0f, 0.34f, 0.61f, 0.577f, -0.577f, 0.577f,
        0.0f, 0.0f, 0.5f, 0.66f, 0.41f, 0.577f, -0.577f, 0.577f,

        // Cara 8: Inferior Trasera-Izquierda (-, -, -)
        0.0f, -0.5f, 0.0f, 0.67f, 0.80f, -0.577f, -0.577f, -0.577f,
        -0.5f, 0.0f, 0.0f, 0.99f, 0.61f, -0.577f, -0.577f, -0.577f,
        0.0f, 0.0f, -0.5f, 0.99f, 0.99f, -0.577f, -0.577f, -0.577f,
    };

    Mesh* octaedro = new Mesh();
    // Son 24 vértices en total, por 8 datos cada uno (x,y,z, s,t, nx,ny,nz) = 192.
}
```

```

        // Y 24 índices que conforman los 8 triángulos.
        octaedro->CreateMesh(octa_vertices, octa_indices, 192, 24);
        meshList.push_back(octaedro); // Este será el índice [5] en tu meshList
    }
    // =====

```

Dentro de nuestro int main mandamos a llamar a la función crearOctaedro y líneas de código más abajo también asignamos nuestra imagen de octaedro.tga para el texturizado.

```

// =====
CrearOctaedro();
// =====
OctaedroTexture = Texture("Textures/OCTAEDRO.tga");
OctaedroTexture.LoadTextureA();

```

Finalmente, dentro de la parte de blending, después del agave, mandamos a renderizar nuestro dado, y aquí también cabe mencionar que dejamos la rotación del mismo, sabemos que si nuestro dado hubiera estado estático quizá las normales pudimos ponerlas con distintos valores para que la parte “trasera del dado” respecto a la luz no reflejara tanto la misma, pero debido a la rotación que mantuvimos, dejamos las normales con la misma magnitud para todas las caras.

```

// AGAVE (BLENDING) -----
// ELEMENTOS CON TRANSPARENCIA SE DIBUJAN AL ULTIMO
// Renderizado al final para correcta transparencia
...
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

AgaveTexture.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[3]->RenderMesh();

// =====
// Renderizado del Octaedro
model = glm::mat4(1.0);
// Lo trasladamos un poco a la derecha para que no choque con tus cubos
model = glm::translate(model, glm::vec3(-10.0f, 5.0f, 6.0f));
model = glm::rotate(model, mainWindow.getRotacionDado() * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
OctaedroTexture.UseTexture();
meshList[5]->RenderMesh();
// =====

glDisable(GL_BLEND);

```

Apagado de una luz

Como segunda actividad se pidió apagar con teclado la luz (pointlight) de su lámpara creada para el reporte de la práctica 7. Para ello se modificaron un total de 3 archivos.

Window

Como ya es muy costumbre, modificamos nuestros window.h y window.cpp para agregar una variable y un getter que controlen este cambio, misma que se decidió hacer booleana debido a que solo fluctúa entre “apagado” o “encendido”

```
bool getApagarLampara() { return apagarLampara; } //Window.h:public
bool apagarLampara; // //Window.h:private
apagarLampara = false; // Window.cpp Inicializando
// 9: Interruptor Lámpara EN WINDOW.cpp: Maneja teclado
if (key == GLFW_KEY_9 && action == GLFW_PRESS) {
    theWindow->apagarLampara = !theWindow->apagarLampara;
}
```

Main

Se siguió lo comentado en clase, para realmente apagar una luz, no hay que enviarla al shader y este cambio solo se da en ese momento, pero al tener un control para ello, hacemos uso del cambio que agregamos en window.cpp

```
// =====
// FASE 2: ENVÍO DE LUCES AL SHADER
// =====
shaderList[0].SetDirectionalLight(&mainLight);
// LÓGICA PARA APAGAR LA LÁMPARA HEXTECH
if (mainWindow.getApagarLampara() == true) {
    // Con la tecla 9 (apagarLampara) decidimos si tenemos nuestro arreglo en 1 o en 0
    shaderList[0].SetPointLights(pointLights, pointLightCount - 1);
}
else {
    // Estado normal, le mandamos el total de luces (1) al shader. La luz brilla.
    shaderList[0].SetPointLights(pointLights, pointLightCount);
}
shaderList[0].SetSpotLights(spotLights, spotLightCount);
// PARA APAGAR ALGUNA LUZ, SOLO PUEDE SER POR AQUI, le debemos poner
// shaderList[0].SetPointLights(pointLights, pointLightCount-1);
// Con esto eliminamos (apagamos) la ultima(s) luz(luces), pero debemos verificar
// que las que ocupamos apagar esten hasta el final del arreglo o sino
// con condicionales cambiar nuestro arreglo activo, como en el previo.
```

Cambio de luces

Como último ejercicio se pidió que, al presionar una tecla, la luz del faro del coche cicle de color entre rojo, amarillo, verde, cian, azul, magenta.

En un inicio se pensó en solo crear un arreglo de vec3 para instanciar distintos colores, pero para dar más dinamismo al ejercicio también se hizo que los valores de apertura del cono y de atenuación variaran con cada luz diferente, justificando así la propuesta dada en clase de crear un arreglo auxiliar para las luces.

Window

Como el cambio se hace mediante teclado nuevamente asignamos un getter para ello, pero este será entero pues se enviara como posición a un arreglo auxiliar el main.

```
int getColorFaroCoche() { return colorFaroCoche; } //Window.h:public
int colorFaroCoche; // //Window.h:private
colorFaroCoche = 5; // Window.cpp Inicializando en magenta
// 0: Ciclar colores del faro del coche EN WINDOW.cpp: Maneja teclado
if (key == GLFW_KEY_0 && action == GLFW_PRESS) {
    theWindow->colorFaroCoche++; // Sumamos 1
    if (theWindow->colorFaroCoche > 5) { // Si pasa de 5 (Magenta), regresa a 0 (Rojo)
        theWindow->colorFaroCoche = 0;
    }
}
```

Main

Dentro de nuestra función int main la declaración de arreglos de luces puntuales se mantuvo, incluyendo la luz del faro azul que ya se tenía, debido a que se sobrescribiera más adelante. Fue entonces que se agregó nuestro arreglo de faros auxiliares

```
// =====
// ARREGLO AUXILIAR DE FAROS (Aqui ponemos distintas luces auxiliares para el faro)
// =====
SpotLight farosAuxiliares[6];
// 0: Faro ROJO (Cono cerrado, llega lejos)
farosAuxiliares[0] = SpotLight(1.0f, 0.0f, 0.0f, // Color Rojo (RGB)
    1.0f, 1.0f, // Intensidades
    0.0f, 0.0f, 0.0f, 1.0f, -0.2f, 0.0f, // Posición y Dirección
    1.0f, 0.045f, 0.0075f, // Atenuación
    30.0f); // Ángulo de apertura
// 1: Faro AMARILLO (Cono medio)
farosAuxiliares[1] = SpotLight(1.0f, 1.0f, 0.0f, // Color Amarillo (RGB)
    1.0f, 1.0f
    0.0f, 0.0f, 0.0f, 1.0f, -0.2f, 0.0f,
    1.0f, 0.045f, 0.0075f,
    45.0f);
// 2: Faro VERDE (Cono amplio, se atenúa más rápido)
farosAuxiliares[2] = SpotLight(0.0f, 1.0f, 0.0f, // Color Verde (RGB)
    1.0f, 1.0f,
    0.0f, 0.0f, 0.0f, 1.0f, -0.2f, 0.0f,
    1.0f, 0.09f, 0.032f,
    60.0f);
// 3: Faro CIAN
farosAuxiliares[3] = SpotLight(0.0f, 1.0f, 1.0f, // Color Cian (RGB)
    1.0f, 1.0f,
    0.0f, 0.0f, 0.0f, 1.0f, -0.2f, 0.0f,
    1.0f, 0.045f, 0.0075f,
    75.0f);
// 4: Faro AZUL (Original)
farosAuxiliares[4] = SpotLight(0.0f, 0.0f, 1.0f, // Color Azul (RGB)
    1.0f, 1.0f,
    0.0f, 0.0f, 0.0f, 1.0f, -0.2f, 0.0f,
    1.0f, 0.045f, 0.0075f,
    90.0f);
// 5: Faro MAGENTA (Cono gigante)
farosAuxiliares[5] = SpotLight(1.0f, 0.0f, 1.0f, // Color Magenta (RGB)
    1.0f, 1.0f,
    0.0f, 0.0f, 0.0f, 1.0f, -0.2f, 0.0f,
    1.0f, 0.022f, 0.0019f,
    120.0f);
```

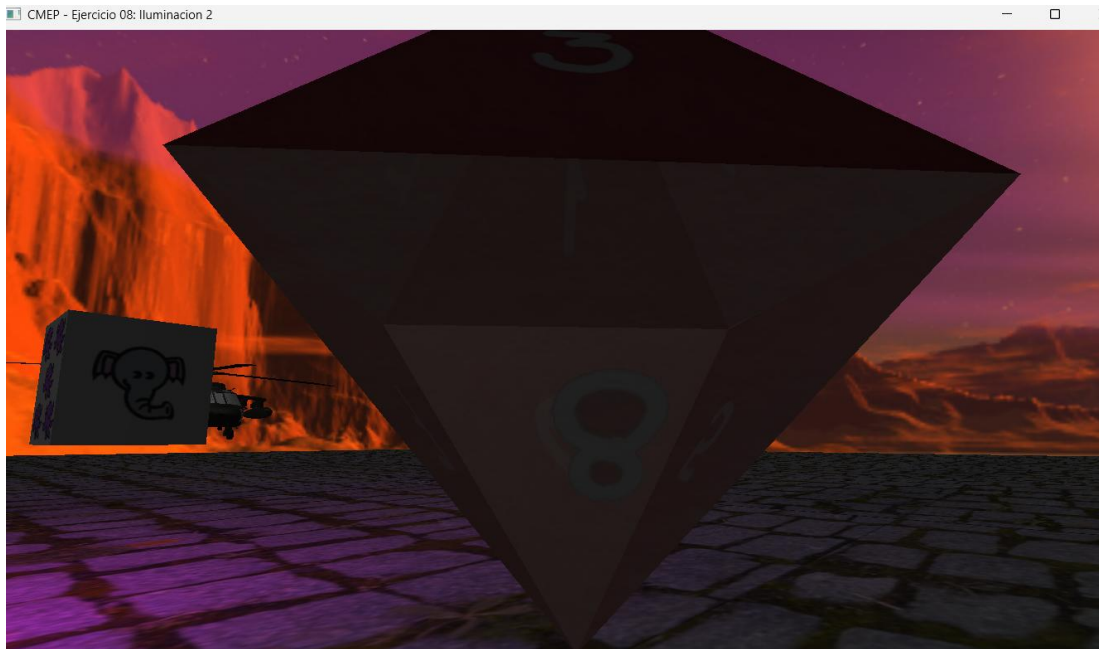
El cambio de estas luces lo hacemos donde declaramos la luz de faro jerarquizada a nuestro coche, todo ello ya dentro de nuestro while por cada ciclo de reloj. Ya que spotLights[1] que es el faro inicialmente azul, se sobrescribe mediante el arreglo auxiliar y mandando al índice el valor que tenemos en el getter de getColorFaroCocdhe.

```
// JERARQUIA Y CAMBIO DE FAROS DEL COCHE (INDICE 1 en spotLights)
// 1. Cambiamos la luz del faro a partir de la tecla 0 (rango [0,5])
spotLights[1] = farosAuxiliares[mainWindow.getColorFaroCoche()];
// 2. Le calculamos su posición global atada a la variable maestra del coche
matrizCocheLuz = glm::mat4(1.0);
matrizCocheLuz = glm::translate(matrizCocheLuz, posCoche); // Usamos la variable maestra
posicionGlobalFaroCoche = matrizCocheLuz * posicionLocalFaroCoche;
// 3. Le actualizamos la posición a nuestra luz recién metida a la cancha
spotLights[1].SetPos(glm::vec3(posicionGlobalFaroCoche));
```

EJECUCIÓN

TECLA	MUEVE	TECLA	MUEVE
1	Cofre (Abre)	2	Cofre (Cierra)
3	Coche (Avanza)	4	Coche (Retrocede)
5	Helicoptero (Avanza)	6	Helicóptero (Retrocede)
7	Gira Dado (Der)	8	Gira dado (Izq)
9	Enciende/Apaga lampara	0	Cambiafaro del coche





Para una mejor visualización se adjunta un link al GIF de drive.

https://drive.google.com/file/d/1vmJ_4givwcox-WO6W6FfG-St_ma3I2Ei/view?usp=sharing

PROBLEMAS PRESENTADOS

No se definía que valores debían agregarse a las normales, ya que al poner valores a prueba y error los resultados no eran convincentes ni deseados y como cambiarían estos, por lo que se optó por preguntarle a una IA sobre la teoría detrás de estos números para un octaedro, obteniendo los siguientes resultados.

Hola. Estoy haciendo un trabajo para mi materia de Computación gráfica C++ con OpenGL. Nos pidieron hacer un dado de 8 caras (un octaedro), cosa que ya hice en un trabajo pasado, solo que ahora nos dieron una instrucción extra de agregar los valores normales (Nx, Ny, y Nz)

El profesor nos dijo que para que la luz funcione bien y el dado se vea sólido, tenemos que configurar estas "normales" en los vértices (Nx, Ny, Nz), y yo tengo entendido que esas normales deben apuntar hacia afuera de las caras de mi octaedro.

Mi duda viene desde algo matemático.

¿Qué números exactos debo poner en Nx, Ny y Nz y por qué? Me parece que debo hacerlo de forma "prueba y error" pero no se si haya alguna explicación algo más teorica de esto para calcular el número exacto y que la luz no se vea débil o con errores.

¿Me podrías explicar paso a paso cómo se calcula matemáticamente ese valor exacto para las caras de mi octaedro?

Te pasaré mi función de crear octaedro para que sepas a que me refiero con valores de Nx, Ny y Nz:

```
//
=====
```

```
=====
=====
void CrearOctaedro()
{
    unsigned int octa_indices[] =
    {
        // --- MITAD SUPERIOR ---
        0, 1, 2, // Cara 1: Superior Frontal-Derecha
        3, 4, 5, // Cara 2: Superior Trasera-Derecha
        6, 7, 8, // Cara 3: Superior Trasera-Izquierda
        9, 10, 11, // Cara 4: Superior Frontal-Izquierda

        // --- MITAD INFERIOR ---
        12, 13, 14, // Cara 5: Inferior Frontal-Derecha
        15, 16, 17, // Cara 6: Inferior Trasera-Derecha
        18, 19, 20, // Cara 7: Inferior Trasera-Izquierda
        21, 22, 23 // Cara 8: Inferior Frontal-Izquierda
    };

    GLfloat octa_vertices[] = {
        // x   y   z   S   T   NX   NY   NZ
        // ===== MITAD SUPERIOR
        // =====
        // Cara 1: Superior Frontal-Derecha
```



```
=====
// Cara 1: Superior Frontal-Derecha
0.0f, 0.5f, 0.0f, 0.32f, 0.20f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, 0.5f, 0.01f, 0.39f, 0.0f, 0.0f, 0.0f,
0.5f, 0.0f, 0.0f, 0.01f, 0.01f, 0.0f, 0.0f, 0.0f,

// Cara 2: Superior Trasera-Derecha
0.0f, 0.5f, 0.0f, 0.33f, 0.21f, 0.0f, 0.0f, 0.0f,
0.5f, 0.0f, 0.0f, 0.33f, 0.59f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, -0.5f, 0.01f, 0.41f, 0.0f, 0.0f, 0.0f,

// Cara 3: Superior Trasera-Izquierda
0.0f, 0.5f, 0.0f, 0.34f, 0.21f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, -0.5f, 0.65f, 0.40f, 0.0f, 0.0f, 0.0f,
-0.5f, 0.0f, 0.0f, 0.34f, 0.59f, 0.0f, 0.0f, 0.0f,

// Cara 4: Superior Frontal-Izquierda
0.0f, 0.5f, 0.0f, 0.35f, 0.21f, 0.0f, 0.0f, 0.0f,
-0.5f, 0.0f, 0.0f, 0.67f, 0.01f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, 0.5f, 0.67f, 0.40f, 0.0f, 0.0f, 0.0f,

// ===== MITAD INFERIOR
=====

// Cara 5: Inferior Frontal-Izquierda
0.0f, -0.5f, 0.0f, 0.66f, 0.80f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, 0.5f, 0.33f, 0.99f, 0.0f, 0.0f, 0.0f,
-0.5f, 0.0f, 0.0f, 0.33f, 0.61f, 0.0f, 0.0f, 0.0f,

// Cara 7: Inferior Trasera-Derecha
0.0f, -0.5f, 0.0f, 0.67f, 0.79f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, -0.5f, 0.67f, 0.41f, 0.0f, 0.0f, 0.0f,
0.5f, 0.0f, 0.0f, 0.98f, 0.60f, 0.0f, 0.0f, 0.0f,

// Cara 6: Inferior Frontal-Derecha
0.0f, -0.5f, 0.0f, 0.66f, 0.79f, 0.0f, 0.0f, 0.0f,
0.5f, 0.0f, 0.0f, 0.34f, 0.61f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, 0.5f, 0.66f, 0.41f, 0.0f, 0.0f, 0.0f,

// Cara 8: Inferior Trasera-Izquierda
0.0f, -0.5f, 0.0f, 0.67f, 0.80f, 0.0f, 0.0f, 0.0f,
-0.5f, 0.0f, 0.0f, 0.99f, 0.61f, 0.0f, 0.0f, 0.0f,
0.0f, 0.0f, -0.5f, 0.99f, 0.99f, 0.0f, 0.0f, 0.0f
};
```

¡Qué onda! Qué buen proyecto para tu materia en la UNAM. El octaedro es el ejemplo perfecto para entender normales porque, a diferencia de una esfera donde las normales cambian suavemente, aquí cada cara es plana y necesita su propio vector para que la luz rebote correctamente y se vea "sólido" (Flat Shading).

Olvidate de la prueba y error. Hay una fórmula matemática exacta para esto basada en el **producto cruz**. Aquí te explico paso a paso cómo calcular esos valores de N_x , N_y , N_z .

1. La Teoría: El Producto Cruz

Para encontrar la normal de una cara triangular, necesitas dos vectores que vivan en esa cara. Si tienes los vértices $V1$, $V2$ y $V3$, puedes crear dos vectores:

- $A = V2 - V1$
- $B = V3 - V1$

La **Normal** (N) es el resultado del producto cruz entre ellos:

$$N = A \times B$$

Importante: El orden importa por la "regla de la mano derecha". Si los vértices están en sentido antihorario (vistos desde afuera), la normal apuntará hacia afuera.

2. Cálculo paso a paso (Ejemplo: Cara 1)

Tomemos los datos de tu código para la **Cara 1** (Superior Frontal-Derecha):

- $V1 = (0.0, 0.5, 0.0)$
- $V2 = (0.0, 0.0, 0.5)$
- $V3 = (0.5, 0.0, 0.0)$

Paso A: Crear los vectores

$$A = V2 - V1 = (0.0 - 0.0, 0.0 - 0.5, 0.5 - 0.0) = (0, -0.5, 0.5)$$

$$B = V3 - V1 = (0.5 - 0.0, 0.0 - 0.5, 0.0 - 0.0) = (0.5, -0.5, 0)$$

Paso B: Producto Cruz

$$N = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 0 & -0.5 & 0.5 \\ 0.5 & -0.5 & 0 \end{vmatrix}$$

- $N_x = (-0.5 \cdot 0) - (0.5 \cdot -0.5) = 0.25$
- $N_y = (0.5 \cdot 0.5) - (0 \cdot 0) = 0.25$
- $N_z = (0 \cdot -0.5) - (-0.5 \cdot 0.5) = 0.25$

$N = (0.25, 0.25, 0.25)$		2	Superior Trasera-Derecha	0.577	0.577
Paso C: Normalización (Vital para OpenGL)		3	Superior Trasera-Izquierda	-0.577	0.577
OpenGL espera que las normales sean vectores unitarios (longitud = 1). Si no lo haces, la luz se verá exageradamente brillante o muy oscura.		4	Superior Frontal-Izquierda	-0.577	0.577
La magnitud es $L = \sqrt{0.25^2 + 0.25^2 + 0.25^2} \approx 0.433$.		5	Inferior Frontal-Izquierda	-0.577	-0.577
Dividimos cada componente entre L :		6	Inferior Frontal-Derecha	0.577	-0.577
$N_x = 0.25/0.433 \approx \mathbf{0.577}$ $N_y = 0.25/0.433 \approx \mathbf{0.577}$ $N_z = 0.25/0.433 \approx \mathbf{0.577}$		7	Inferior Trasera-Derecha	0.577	-0.577
3. Atajo Simétrico		8	Inferior Trasera-Izquierda	-0.577	-0.577
Como un octaedro es totalmente simétrico y tus vértices están en 0.5 o -0.5, todas tus normales tendrán los mismos números, solo cambiarán los signos dependiendo del cuadrante.		Exportar a Hojas de cálculo			
Aquí tienes la tabla de valores que debes poner en tu arreglo <code>octo_vertices</code> :		Ojo con el código: En tu código la Cara 5 dice "Inferior Frontal-Izquierda" y la Cara			
Cara	Descripción	Nx	Ny		
1	Superior Frontal-Derecha	0.577	0.577		

Ojo con el código: En tu código la Cara 5 dice "Inferior Frontal-Izquierda" y la Cara 6 "Inferior Frontal-Derecha". Solo asegúrate de que los signos de N_x y N_z coincidan con los signos de los vértices de la base de esa cara, y N_y siempre será negativo en la mitad inferior.

Estoy buscando libros de texto de Gráficos por Computadora, documentación oficial de OpenGL, o artículos técnicos reconocidos (como de universidades o sitios especializados) que hablen específicamente sobre la "normalización de vectores de vértice" y el "Flat Shading" para poder agregarlos a mis referencias.

¿Por qué esto hace que se vea sólido?

Al darle la misma normal a los 3 vértices de un triángulo, le dices a OpenGL: "Esta superficie es plana y apunta hacia allá". Si compartieras vértices (usando solo 6 vértices en lugar de 24), OpenGL promediaria las normales y el dado se vería "redondeado" o suavizado, lo cual no queremos para un dado.

¡Mucho éxito con ese proyecto de la Facultad de Ingeniería! Si necesitas ayuda con el sombreador de fragmentos para aplicar esto, me avisas.

👍 🔄 📄 ⋮

Me quedó claro lo de calcular la longitud del vector para que valga 1 y por qué el resultado da 0.577. Como tengo que entregar un reporte formal detallando esta práctica, necesito justificar esta decisión técnica en mi bibliografía, entonces... ¿Me podrías compartir de dónde sacaste esta información?

Para tu reporte académico en la UNAM, es fundamental citar las fuentes "canónicas" de la computación gráfica. La necesidad de normalizar vectores y el uso de normales por cara para lograr el Flat Shading (sombreado plano) se basa en el **Modelo de Reflexión de Phong** y en la arquitectura de la canalización (pipeline) de OpenGL.

Aquí tienes las referencias bibliográficas más sólidas y reconocidas:

1. El "Libro de Oro" de la Computación Gráfica

Esta es la referencia obligada en cualquier curso universitario.

- **Referencia:** Hearn, D., & Baker, M. P. (2011). *Computer Graphics with OpenGL*. Pearson Education.
- **Justificación:** En el capítulo de "Illumination Models and Surface-Rendering Methods", explican que para superficies poligonales planas, el vector normal debe ser perpendicular al plano del polígono. También detallan el **Flat Shading**, donde se asigna una única normal a toda la cara para evitar la interpolación de luz y mantener el aspecto facetado (como un dado).

2. Documentación Oficial de OpenGL (The Blue Book)

Si necesitas citar la fuente técnica directa de la API que estás usando.

- **Referencia:** Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL Programming Guide: The Official Guide to Learning OpenGL*. Addison-Wesley Professional.
- **Justificación:** Específicamente en las secciones sobre *Vertex Data* y *Lighting*, se menciona que las normales deben ser **vectores unitarios** (normalizados) porque las ecuaciones de iluminación (como el producto punto $L \cdot N$) asumen que la magnitud del vector es 1. Si no se normalizan, el cálculo de la intensidad luminica falla.

3. El Modelo de Iluminación de Phong (Artículo Clásico)

Si quieres irte a la raíz matemática de por qué la normal es tan importante para la luz.

- **Referencia:** Phong, B. T. (1975). "Illumination for computer generated pictures". *Communications of the ACM*, 18(6), 311-317.
- **Justificación:** Este es el artículo original donde se explica cómo el ángulo entre la fuente de luz y la **normal de la superficie** determina el brillo de un píxel.

4. Recurso Web Especializado (Referencia Moderna)

Muy útil para explicar la implementación práctica en C++.

- **Referencia:** De Vries, J. (2014). *Learn OpenGL: Normal Mapping*. Recuperado de learnopengl.com.
- **Justificación:** Es el sitio de referencia actual para desarrolladores. Explica visualmente por qué las normales deben ser perpendiculares a los fragmentos para calcular correctamente la reflexión difusa.

Es importante mencionar que se comprobó la respuesta y la información que se dio UNICAMENTE en las referencias 1 y 3, de las cuales se adjunta el link en la sección de referencias. La referencia 2 no se encontró la parte específica de donde encontró o extrajo información (es un libro demasiado largo que al menos al probar, no arrojo resultados de palabras clave de luces) mientras que la referencia 4 ya se había consultado anteriormente pero por el ejemplo de la pared de ladrillos que ponen se descartó para este ejercicio en específico y lo que se quiere lograr, pues se entendió que esa sección era más destinada a “engañar” a la luz a partir de una superficie plana para hacer parecer que cierto plano tiene profundidad.

CONCLUSIONES

Sobre el ejercicio de clase

Hacer esta práctica me ayudó a entender mucho mejor cómo las matemáticas afectan lo que vemos en pantalla. Calcular a mano los vectores normales del octaedro sinceramente no se comprendió por completo al inicio al ser prueba y error, pero me sirvió para entender que no basta con solo indicar hacia dónde apunta una cara; el vector tiene que estar normalizado (medir exactamente 1, por eso usamos el valor de 0.577). Si no hacíamos esto, el cálculo de la luz fallaba y el llamado "Flat Shading" se veía mal.

Por otro lado, al meter la transparencia en el dado, me di cuenta de lo importante que es el Z-buffer y el orden en que dibujamos los polígonos. Apagar el Face Culling temporalmente fue una opción que se consideró pero no se notó algún cambio, por lo que al final se siguió la sugerencia del profesor de instanciar las caras transparentes al final, aunque considero que no es la solución más idónea ni universal para otros casos.

En cuanto a las luces, aprendí que la forma correcta y optimizada de "apagar" una lámpara no es bajarle la intensidad a cero como lo había hecho en el ejercicio pasado, sino simplemente restarla del contador para no enviarla al Shader. Además, para el faro del coche, usar un arreglo auxiliar de luces para ir cambiando de color y de ángulo de apertura resultó ser una solución muy limpia que no satura la memoria. Finalmente, acomodar el código para que las luces y los modelos compartieran la misma variable de posición (jerarquías) hizo que mover el coche y el helicóptero fuera mucho más fácil y ordenado.

Comentarios generales

Para esta práctica decidí mapear todas las nuevas animaciones (cofre, llantas, helicóptero y dado) a los números del 1 al 8, lo cual hace que probar la escena sea mucho más cómodo.

Como sugerencia para futuras prácticas o para el proyecto final que involucren modelos externos, sería de gran ayuda que nos pudieran compartir enlaces a tutoriales cortos de Blender especialmente de cómo se ven las normales en ese software en caso de que se decida volver a compartir material de video externo.

Referencias y bibliografía

- **Hearn, D., Baker, M. P., & Carithers, W. R. (2013).** *Computer Graphics with OpenGL* (4ta ed.). Pearson Education. Recuperado el 11 de abril de 2026, de [https://drive.uqu.edu.sa/mskhayat/files/MySubjects/2019SumS_ComputerGraphics/Computer%20Graphics%20with%20OpenGL%20\(4th%20ed\)%20%5BHearn,%20Baker%20&%20Carithers%202013%5D.pdf](https://drive.uqu.edu.sa/mskhayat/files/MySubjects/2019SumS_ComputerGraphics/Computer%20Graphics%20with%20OpenGL%20(4th%20ed)%20%5BHearn,%20Baker%20&%20Carithers%202013%5D.pdf)
- **Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013).** *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3* (8va ed.). Addison-Wesley Professional. Recuperado el 11 de abril de 2026, de

<https://www.cs.utexas.edu/~fussell/courses/cs354/handouts/Addison.Wesley.OpenGL.Programming.Guide.8th.Edition.Mar.2013.ISBN.0321773039.pdf>

- **Phong, B. T. (1975).** *Illumination for computer generated pictures*. Communications of the ACM, 18(6), 311-317. Recuperado el 11 de abril de 2026, de https://users.cs.northwestern.edu/~ago820/cs395/Papers/Phong_1975.pdf
- **Google Gemini. (2026).** *Interacción de chat con IA sobre cálculo de normales y sombreado plano en OpenGL*. Recuperado el 11 de abril de 2026, de <https://gemini.google.com>